

自注意力机制（下）

李宏毅《机器学习》2025 第四节：矩阵化、多头、位置与应用边界



基于李宏毅老师课程整理

2026 年 6 月 8 日

视频信息

频道：李宏毅机器学习深度学习课程（Bilibili）

时长：约 46 分钟

链接：<https://www.bilibili.com/video/BV1TAtwzTE1S?p=13>

目录

1 从一个输出到整排输出	3
1.1 本章小结	4
2 矩阵化写法	4
2.1 把向量拼成矩阵	4
2.2 哪些是参数，哪些只是操作	6
2.3 本章小结	6
3 Multi-head Self-attention	6
3.1 本章小结	7
4 位置资讯从哪里来	7
4.1 本章小结	8
5 语音：序列太长时怎么办	9
5.1 本章小结	9
6 图像：Self-attention 与 CNN	10
6.1 图像也可以是一组向量	10
6.2 CNN 是简化版 Self-attention?	10
6.3 本章小结	12
7 RNN、Graph 与更多变体	12
7.1 与 RNN 的差异	12
7.2 Graph 上的 Self-attention	13
7.3 更多 Efficient Transformer	14
7.4 本章小结	14
8 总结与延伸	15
8.1 延伸阅读	15

1 从一个输出到整排输出

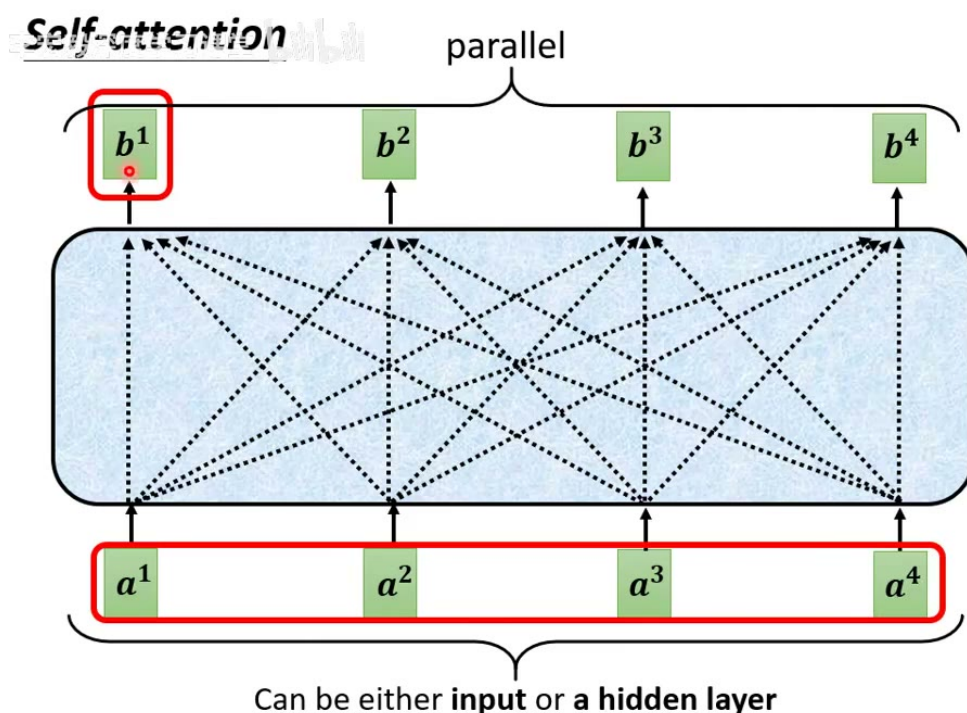
上一讲已经说明：给定输入向量序列

$$(a^1, a^2, \dots, a^N),$$

self-attention 会对每个位置产生一个带上下文的向量

$$(b^1, b^2, \dots, b^N).$$

上半讲为了便于理解，只从 a^1 如何得到 b^1 讲起。下半讲的第一件事，是把同样过程推广到所有位置： b^1 、 b^2 、一直到 b^N 可以同时算出来。



19

图 1: Self-attention 对每个输入位置产生一个输出位置；所有输出可以并行计算，而不是像 RNN 那样逐步推进。²

以第二个输出为例，计算 b^2 时，主角变成 a^2 。模型先由 a^2 产生 query q^2 ，再用它和所有 key 比较：

$$\alpha_{2,j} = q^2 \cdot k^j, \quad j = 1, \dots, N.$$

对这一排分数做 softmax 得到 $\alpha'_{2,j}$ ，再对所有 value 加权求和：

$$b^2 = \sum_j \alpha'_{2,j} v^j.$$

同理， a^3 产生 q^3 ，得到 b^3 ； a^4 产生 q^4 ，得到 b^4 。每个位置的 query 不同，所以每个位置“看其他位置的方式”也不同。

²视频画面时间区间：约 00:30–01:05；字幕对应“怎么从这一排 vector 得到 b two”“b one 跟 b four 是一次同时被计算出来的”。

并行不是说没有相互作用

Self-attention 中每个 b^i 都参考了所有输入位置，所以它们彼此不是独立的；“并行”指的是计算图可以同时展开。所有 query、key、value 都可以先算好，再通过矩阵乘法一次得到整张 attention score matrix。

1.1 本章小结

Self-attention 的每个输出向量都来自全序列加权汇总。它不像 RNN 必须按时间步串行更新状态，因此天然适合用矩阵乘法并行化。

2 矩阵化写法

2.1 把向量拼成矩阵

课程采用“每个向量是一列”的记号。设输入矩阵为

$$I = [a^1, a^2, \dots, a^N].$$

则所有 query、key、value 可以一次性写成

$$Q = W^q I, \quad K = W^k I, \quad V = W^v I.$$

这里 Q 的第 i 列是 q^i ， K 的第 i 列是 k^i ， V 的第 i 列是 v^i 。

接下来，所有原始 attention scores 可以由一次矩阵乘法得到：

$$A = K^\top Q.$$

在这个 convention 下， A 的第 i 列记录“第 i 个 query 对所有 key 的分数”。对每一列做 softmax，得到归一化后的 attention matrix A' 。最后输出矩阵为

$$O = V A',$$

其中 O 的第 i 列就是输出向量 b^i 。

Self-attention

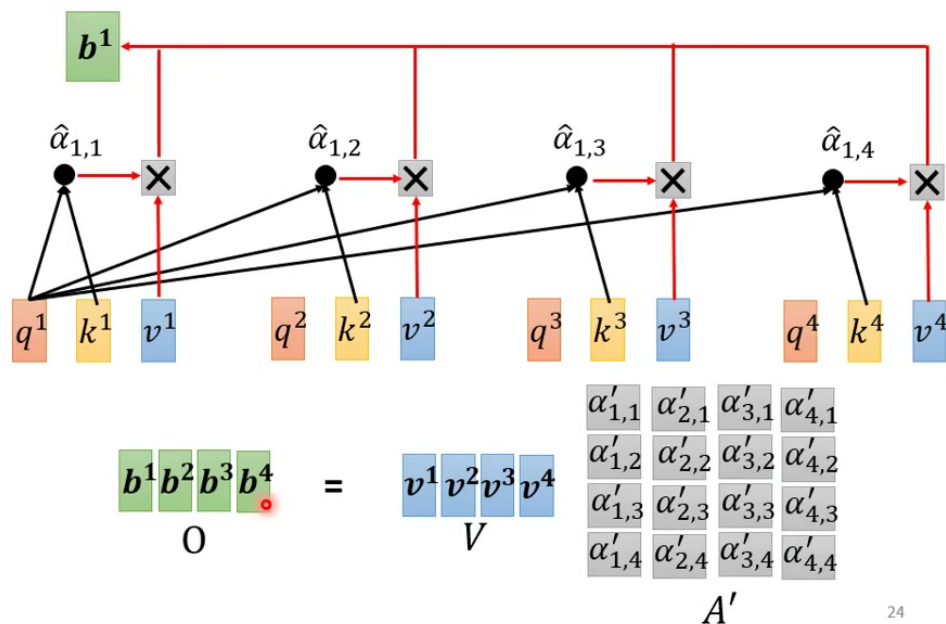


图 2: 把每个位置的 weighted sum 整理成矩阵乘法: 输出矩阵 O 可以写成 VA' 。³

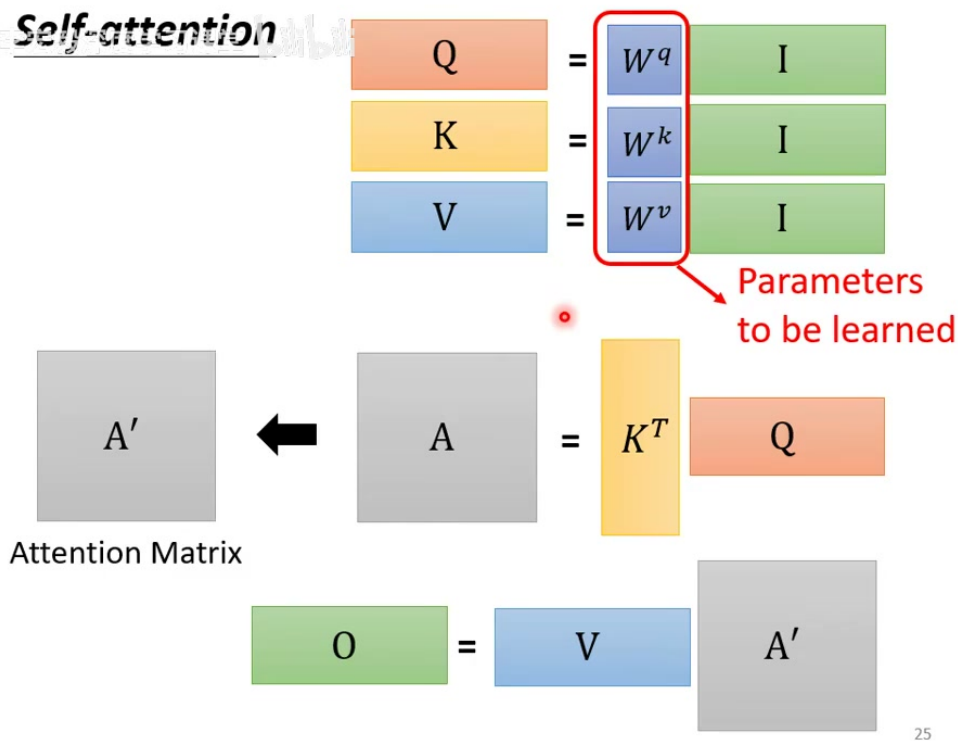


图 3: Self-attention 的矩阵总览: $Q = W^q I$, $K = W^k I$, $V = W^v I$, $A = K^T Q$, $O = V A'$; 需要学习的是投影矩阵。⁵

³视频画面时间区间: 约 12:20–13:10; 字幕对应 “把 A prime 的 column 乘上 V” “得到 b one、b two”。

为什么有些资料写成 $\text{softmax}(QK^T)V$

这只是矩阵排布 convention 不同。课堂中把向量当作 column, 因此写成 $A = K^T Q, O = V A'$; 很多深度学习库把 token 放在 row 上, 因此常写成 $\text{softmax}(QK^T)V$ 。二者表达的是同一件事: query 和 key 算分数, softmax 后加权 value。

2.2 哪些是参数, 哪些只是操作

上面的矩阵式写法容易让人误以为每一步都有新参数。实际上, 最基础的 self-attention 中需要学习的是 W^q, W^k, W^v , 以及后续常见的输出投影矩阵。 $K^T Q$ 、softmax、 $V A'$ 本身只是固定运算, 不额外引入可学习参数。

这点很重要: self-attention 的表达能力来自把输入投影到合适的 query/key/value 空间, 而不是来自手工指定哪个 token 该看哪个 token。训练资料通过损失函数推动这些投影矩阵学会对任务有用的相关性。

2.3 本章小结

矩阵化写法把逐位置的 attention 统一成三个步骤: 线性投影得到 Q/K/V, 矩阵乘法得到 attention matrix, value 与归一化权重相乘得到输出。

3 Multi-head Self-attention

单头 attention 只用一组 query/key/value 空间来定义相关性。但“相关”可能有很多种含义。对文字来说, 一个词可能需要关注主语、宾语、修饰语、指代对象; 对图像来说, 一个 patch 可能需要关注边缘、颜色区域、远处相似纹理; 对语音来说, 一个 frame 可能需要关注局部音素, 也可能需要参考较远的韵律结构。

Multi-head self-attention 的想法就是: 不要只让模型学习一种相关性, 而是并行学习多种相关性。对第 h 个 head, 可以写成

$$q^{i,h} = W^{q,h} a^i, \quad k^{i,h} = W^{k,h} a^i, \quad v^{i,h} = W^{v,h} a^i.$$

每个 head 独立计算自己的 attention 输出 $b^{i,h}$ 。最后把多个 head 的结果串接起来, 再乘上输出矩阵:

$$b^i = W^O [b^{i,1}; b^{i,2}; \dots; b^{i,H}].$$

⁵视频画面时间区间: 约 14:20–14:55; 字幕对应“唯一需要学的参数只有 WQ, WK, WV ”。

Multi-head Self-attention Different types of relevance

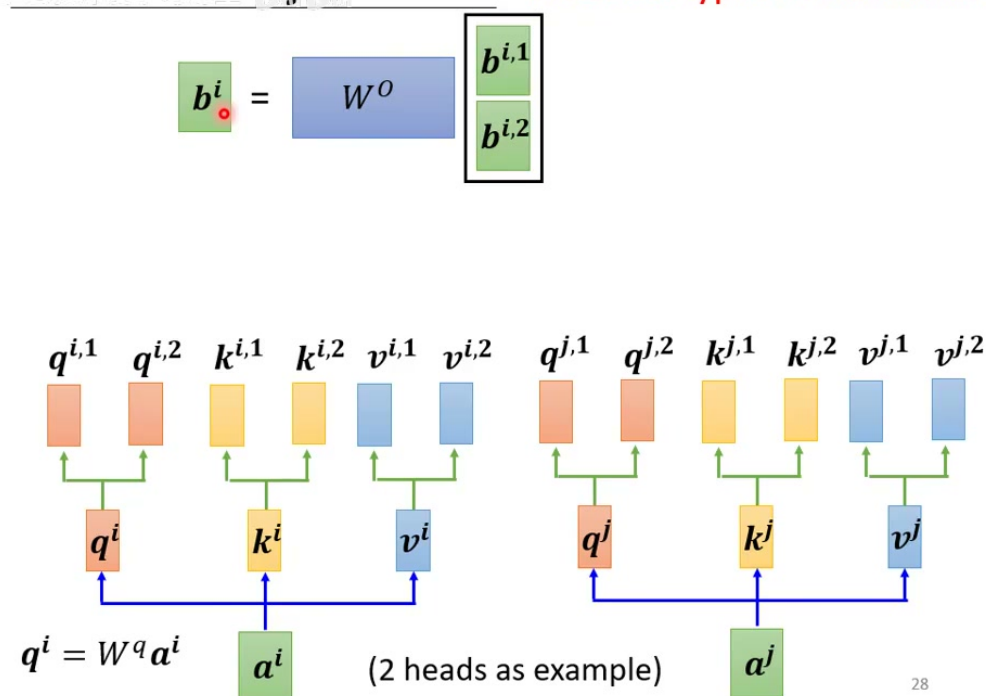


图 4: Multi-head self-attention: 不同 head 各自计算一种相关性，输出串接后再经过 W^O 投影。⁶

Multi-head 的直觉

单头 attention 像只用一种问题去问所有位置：“谁和我相关？” Multi-head 则允许模型同时问多种问题：谁提供句法线索，谁提供语义线索，谁提供局部形状，谁提供全局关系。最后再把这些答案合并成一个新的表示。

3.1 本章小结

Multi-head 不是简单把模型变大，而是让 attention 在多个子空间中分别计算关系。它提升的是“相关性类型”的多样性。

4 位置资讯从哪里来

Self-attention 本身没有内建位置概念。若只看 Q, K, V 的计算，模型对输入顺序并不敏感：把输入向量重新排列，输出也会跟着同样排列，但注意力机制本身不知道“第一个词”“第二个词”这种绝对位置，也不知道两个 token 的距离。

这对很多任务是问题。语言中“狗咬人”和“人咬狗”的词集合相同，但顺序改变会改变意义。语音、图像 patch、程序代码也都需要位置信息。

解决方法是加入 positional encoding。对第 i 个位置设置一个位置向量 e^i ，把它和输入表示结

⁶视频画面时间区间：约 18:35–19:15；字幕对应“如果你有多个 head，有八个 head，有十六个 head，也是一样的操作”。

合，例如

$$\tilde{a}^i = a^i + e^i.$$

然后用 $\tilde{a}^i$ 去产生 query、key、value。这样模型在计算 attention 时就能间接感知位置。

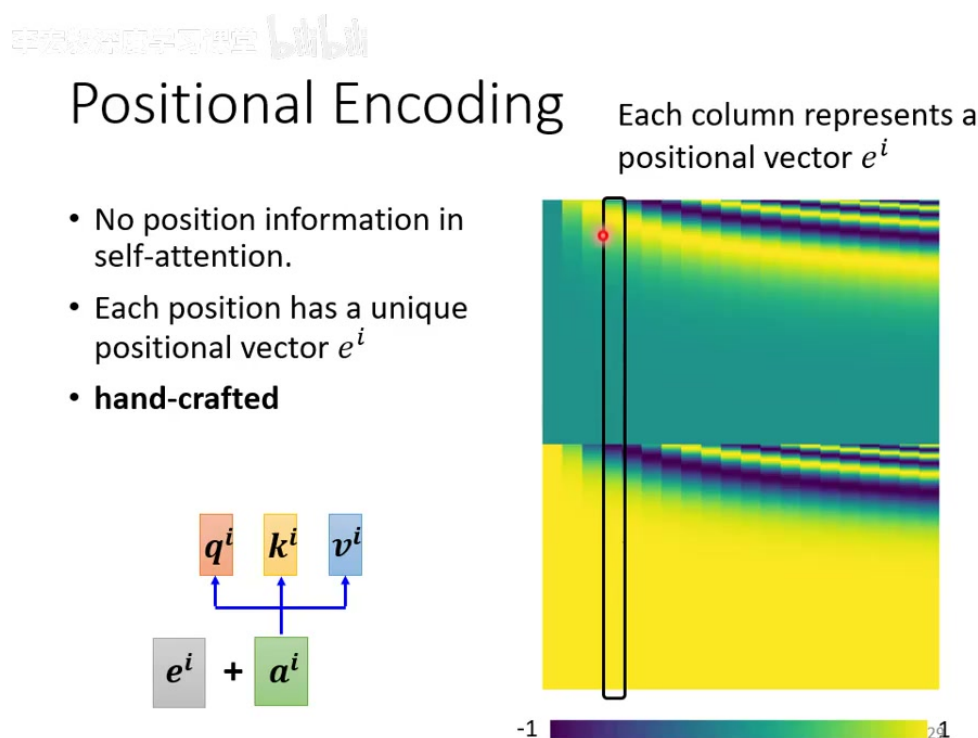


图 5: Positional encoding: self-attention 本身不含位置资讯，因此给每个位置加入唯一的位置向量 e^i 。⁸

最早的 Transformer 使用手工设计的 sinusoidal positional encoding。它的好处是无需为每个位置训练独立参数，并且可以外推到训练中未必见过的长度；坏处是这种设计并不一定适合所有任务。后来有很多改进，例如 learned positional embedding、relative position encoding、旋转位置编码等。课堂没有展开这些细节，但指出了关键事实：positional encoding 本身是 self-attention 架构的重要补丁。

没有位置，attention 不是完整的序列模型

裸 self-attention 只知道“这一批向量彼此如何相关”，不自动知道它们原本排在哪里。处理语言、语音、图像 patch 等有顺序或空间结构的资料时，必须用某种方式把位置信息注入模型。

4.1 本章小结

Self-attention 负责学习内容之间的相关性；positional encoding 负责告诉模型这些内容在序列或空间中的位置。二者合在一起，才构成可用于序列建模的基础模块。

⁸视频画面时间区间：约 22:50–23:20；字幕对应“positional vector 是 hand crafted”“人设的 vector 有很多问题”。

5 语音：序列太长时怎么办

Self-attention 的代价与序列长度强相关。若输入长度为 L ，attention matrix 是 $L \times L$ 。语音讯号尤其容易变长：若每 10ms 产生一个 frame，一秒钟就有约 100 个向量，几秒钟语音就可能产生数百到上千个向量。

因此，对语音直接做全局 self-attention 会遇到计算与记忆体压力。课程提到一种做法：truncated self-attention，也就是只让某个位置在有限范围内 attend，而不是看完整句。

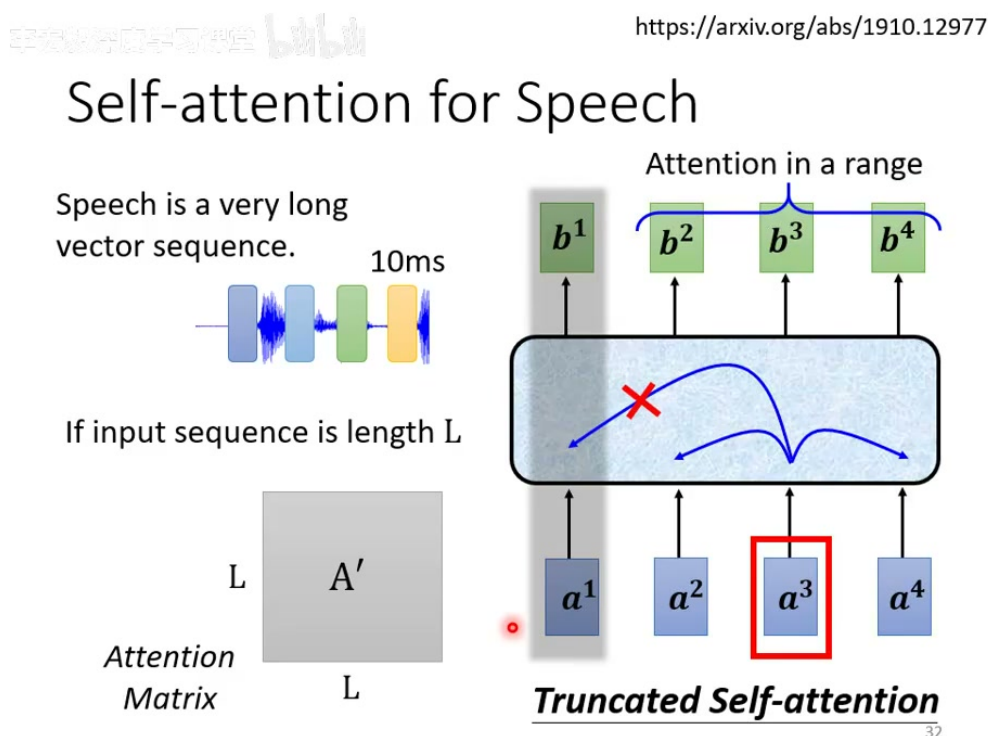


图 6: 语音序列很长，完整 attention matrix 可能过大；truncated self-attention 限制每个位置只看一定范围。¹⁰

这种限制牺牲一部分全局自由度，换来可训练性和效率。它和 CNN 的局部 receptive field 有相似之处：都不是让每个位置自由连接所有位置，而是先把注意范围限制在一块区域内。不同的是，truncated self-attention 在局部范围内仍然是由注意力权重动态选择，而不是固定卷积核。

5.1 本章小结

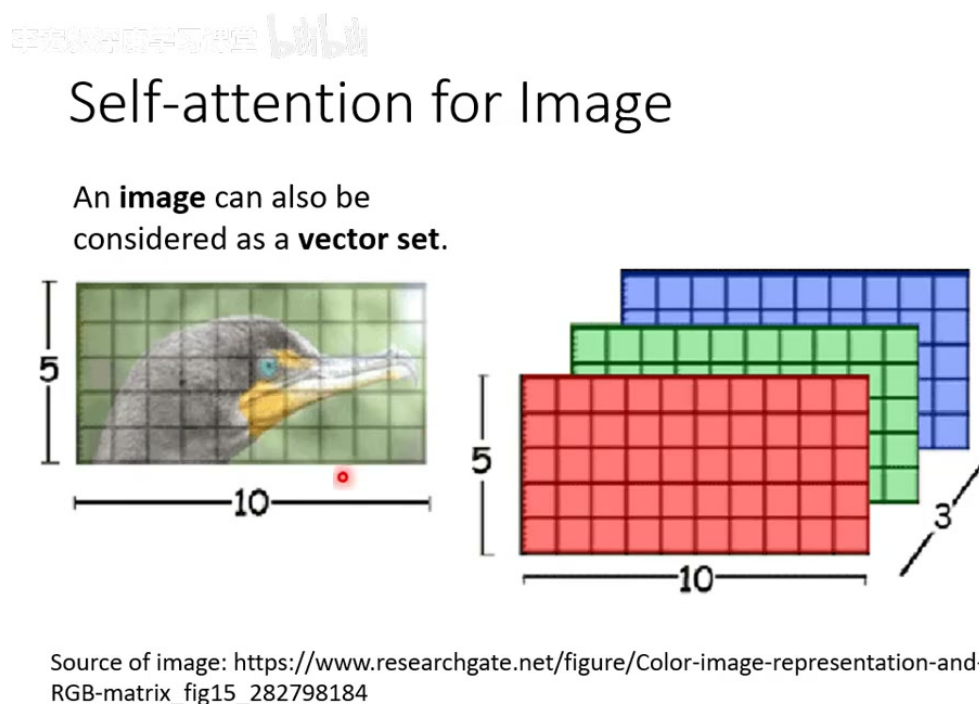
Self-attention 的全局连接很强，但 $O(L^2)$ 的 attention matrix 会成为长序列瓶颈。语音任务常需要对注意范围做裁剪或使用更高效的 attention 变体。

¹⁰ 视频画面时间区间：约 26:50–27:25；字幕对应“attention matrix 可能会太大”“truncated self attention”。

6 图像：Self-attention 与 CNN

6.1 图像也可以是一组向量

图像看似不是序列，但它可以被重写成 vector set。最细的做法是把每个 pixel 当作一个向量；更常见的做法是把图像切成 patch，每个 patch 变成一个 token。这样，图像就能交给 self-attention 处理。



33

图 7：图像可以被视为 vector set：每个 pixel 或 patch 都可以成为一个输入向量。¹¹

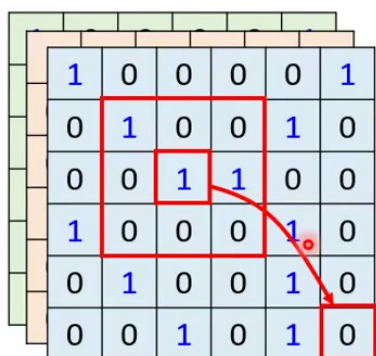
这解释了 Vision Transformer 的基本入口：先把图像切成 patch，把 patch embedding 当作 token sequence，再用 self-attention 建模 patch 之间的关系。

6.2 CNN 是简化版 Self-attention ?

课程给出一个很有启发性的对比：CNN 可以被看成一种受限制的 self-attention，self-attention 也可以被看成一种更复杂、更弹性的 CNN。

¹¹ 视频画面时间区间：约 28:05–29:05；字幕对应“影像这个东西其实也是一个 vector set”。

Self-attention v.s. CNN



CNN: self-attention that can only attends in a receptive field

➤ CNN is simplified self-attention.

Self-attention: CNN with learnable receptive field

➤ Self-attention is the complex version of CNN.

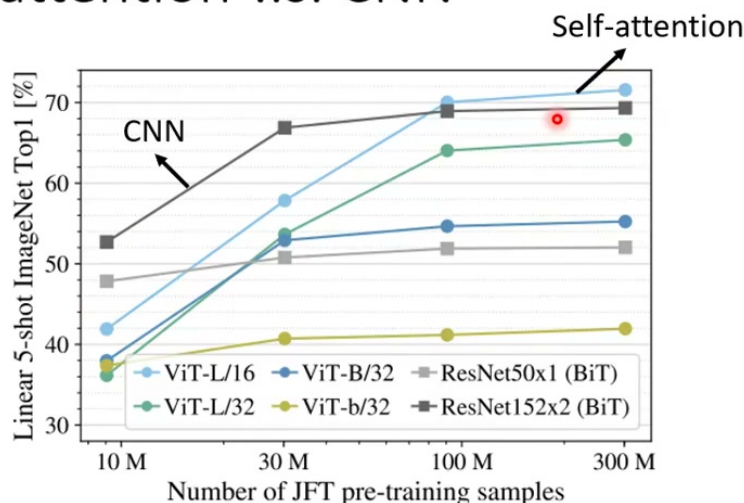
35

图 8: CNN 的 receptive field 由人设定; self-attention 可以学习每个位置该看哪些位置, 因此像是有 learnable receptive field 的 CNN。¹³

CNN 的优势是强 inductive bias: 局部连接、权重共享、平移等变性都写死在结构里。这样的限制让 CNN 在数据不多时更容易学到图像任务中常见的规律。Self-attention 更 flexible, 它可以允许一个位置 attend 到图像中很远的区域, 等于 receptive field 的形状由模型自己学习; 但自由度更高也意味着它通常需要更多数据或更强正则化。

¹³视频画面时间区间: 约 30:35–31:15; 字幕对应 “CNN 是简化版 self attention” “self attention 是复杂化的 CNN”。

Self-attention v.s. CNN



An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale

<https://arxiv.org/pdf/2010.11929.pdf>

图 9: Vision Transformer 与 CNN 的数据规模对比: 数据较少时 CNN 的偏置更有利, 数据足够大时 self-attention 的弹性更容易发挥。¹⁵

弹性与资料量的交换

CNN 把“局部 pattern 很重要”这个先验写进网络, 所以它少学了一些不必要的自由度。Self-attention 更愿意让数据自己决定哪里相关, 因此在大数据下可能更强, 但小数据下也更容易因为自由度过高而吃亏。这不是谁绝对优越, 而是 inductive bias 与数据规模之间的取舍。

6.3 本章小结

图像可以转成 patch sequence 后交给 self-attention。与 CNN 相比, self-attention 的 receptive field 更可学习、更全局, 但也更依赖数据规模和训练技巧。

7 RNN、Graph 与更多变体

7.1 与 RNN 的差异

RNN 处理序列时, 一般沿时间步更新 hidden state。第 t 步的输出依赖前面状态, 因此天然就是串行的。若要让右侧 token 使用左侧很远的信息, 信息必须经过一连串状态传递; 即使使用双向 RNN, 也仍然存在串行计算和长距离依赖路径的问题。

Self-attention 则让任意两个位置通过一层 attention 直接相连, 并且所有位置的输出可并行计算。

¹⁵ 视频画面时间区间: 约 33:35–34:20; 字幕对应“横轴是训练影像的量”“随着资料量越来越多 self attention 的结果变好”。

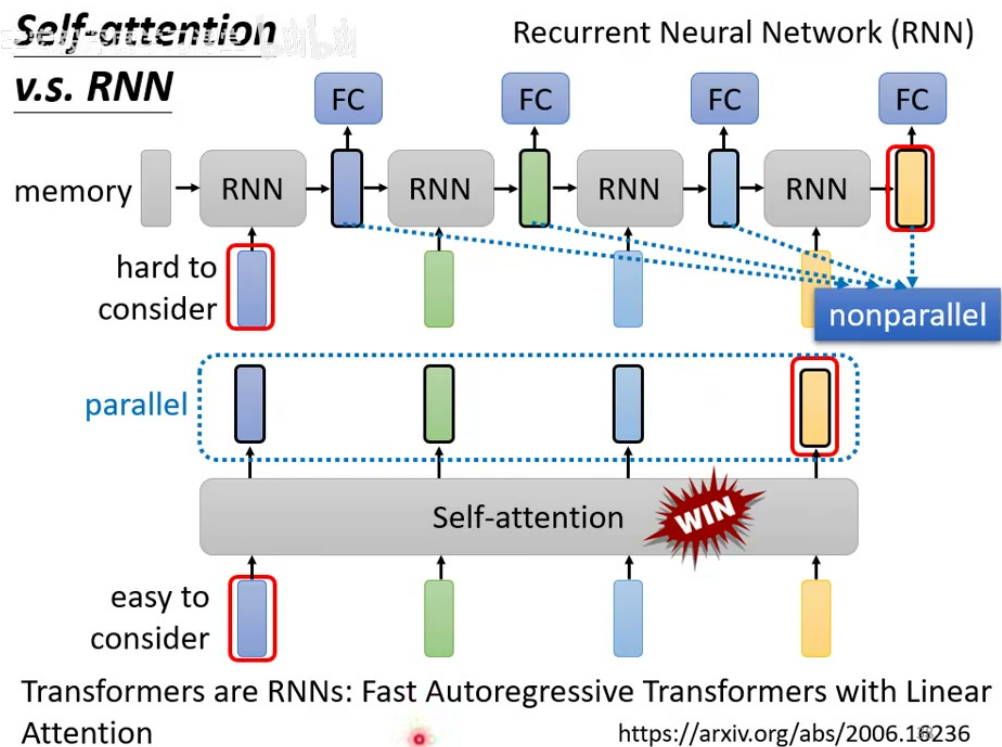


图 10: Self-attention 与 RNN 的对比: self-attention 更容易直接考虑远距离位置, 并且可以并行产生所有输出。¹⁷

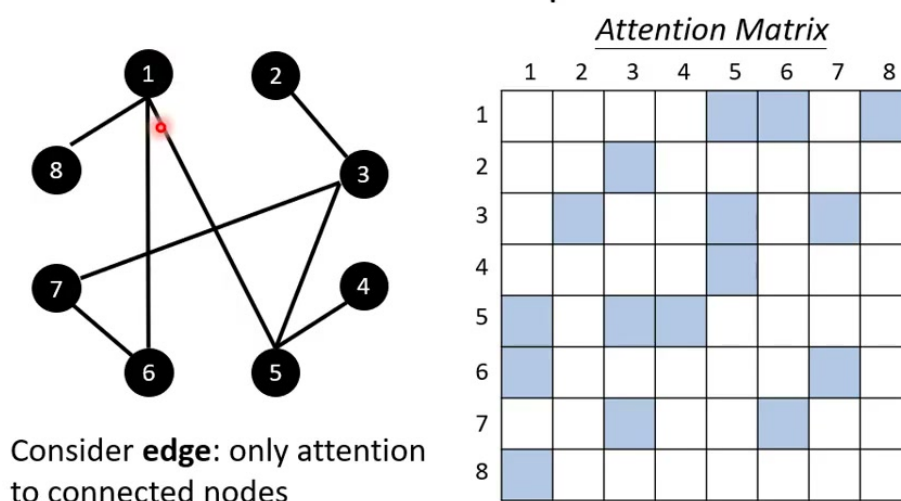
这不是说 RNN 没有价值。RNN 的递推结构本身就是一种强先验, 且在某些流式、低延迟、状态压缩场景中仍有意义。课程此处强调的是: 对标准离线序列建模, self-attention 的并行性和短路径依赖给了它很强的工程优势。

7.2 Graph 上的 Self-attention

Graph 中除了节点向量, 还有边的信息。若完全照搬序列 self-attention, 每个节点都和所有节点计算相关性; 但 graph 已经告诉我们哪些节点相连。因此一种自然做法是只在有边的节点之间计算 attention, 或者用 adjacency mask 限制 attention matrix。

¹⁷ 视频画面时间区间: 约 39:20–40:00; 字幕对应 “每一个 vector 都是同时产生出来的” “self attention 会比 RNN 更有效率”。

Self-attention for Graph



40

图 11: Graph self-attention: 利用边限制 attention, 只在相连节点之间计算或保留注意力分数; 这与 GNN 思想相连。¹⁹

从这个角度看, Graph Neural Network 可以视为在图结构上进行局部信息聚合; graph attention 则让聚合权重由节点特征动态决定。这里的 attention matrix 往往是稀疏的, 因为 graph 本身提供了连接约束。

7.3 更多 Efficient Transformer

课程结尾提到许多名字带有 former 的变体, 例如 Linformer、Performer、Reformer 等。它们大多试图解决同一个根问题: 标准 self-attention 的 $O(N^2)$ 复杂度在长序列上太贵。不同方法可能通过低秩近似、核函数近似、局部窗口、稀疏模式、哈希或递推结构来降低成本。

不要只记“某某 former”的名字

这些变体的共同背景比名字更重要: 标准 self-attention 很强, 但长序列成本高; 因此研究者不断寻找能保留全局建模能力、同时降低计算量的近似或结构化方法。学到这里, 先抓住这个问题意识即可。

7.4 本章小结

Self-attention 相比 RNN 更并行、相比 CNN 更弹性、在 graph 上可被边结构约束。它的强大来自全局交互和可学习相关性, 它的代价则主要是长序列上的平方复杂度。

¹⁹视频画面时间区间: 约 41:20–42:15; 字幕对应“有了 graph 的资讯”“只在相连节点之间计算 attention 的分数”。

8 总结与延伸

P13 把 self-attention 从直观公式推进到现代 Transformer 模块所需的关键组件。

第一，矩阵化使 self-attention 成为高效可并行的层。用列向量记号，它可以压缩为

$$Q = W^q I, \quad K = W^k I, \quad V = W^v I, \quad A = K^\top Q, \quad O = V A'.$$

其中 A' 是对 attention scores 归一化后的矩阵。

第二，multi-head 解决“相关性不止一种”的问题。不同 head 在不同子空间里计算注意力，再把结果合并，使模型能够同时捕捉多类关系。

第三，positional encoding 解决“裸 attention 不知道位置”的问题。没有位置资讯，self-attention 只能处理向量之间的内容相关性，不能区分顺序或空间布局。

第四，应用边界来自任务结构和计算复杂度。语音太长，需要裁剪或高效 attention；图像可以用 patch 建模，但 CNN 的 inductive bias 在小数据时很有价值；RNN 串行但有状态先验；graph 可以用边结构限制 attention。

一句话收束

Self-attention 不是单一公式，而是一种把“元素之间的可学习关系”写进网络的通用机制。Transformer 的力量来自 attention、multi-head、position、feed-forward、normalization 等模块共同组合；而不是仅仅来自“让所有 token 两两相看”。

8.1 延伸阅读

继续往后看 Transformer 时，建议特别关注三件事：残差连接与 layer normalization 如何稳定深层堆叠；encoder-decoder 架构中 self-attention 与 cross-attention 的差异；以及实际训练中为什么需要 mask、dropout、learning rate schedule 等工程技巧。